

Diagrams as a Service

Forschungsbericht

Henri Heyden

stu240825

Christian-Albrechts-Universität zu Kiel

31. März 2026

1 Einleitung

Dieses Semester habe ich in der Eingebettete- und Echtzeitsysteme (RTSys) AG geforscht zum Thema *Diagrams as a Service*. Die RTSys AG beschäftigt sich unter anderem damit eine synchrone Programmiersprache namens *SCCharts* zu entwickeln, welche verwendet wird, um Echtzeitsysteme zu programmieren. *SCCharts* sind ähnlich wie Automaten, weswegen die Entwicklung in dieser Sprache generell graphisch gestützt wird durch eine automatische Visualisierung. Somit befasst sich die RTSys AG viel mit dem Themenbereich der automatischen Synthese und des Renderings von Diagrammen basierend auf Programmiersprachen. Das KIELER Framework bietet einen Language Server der eine Diagramm Synthese zur Verfügung stellt für *SCCharts* und grundlegende Graphenmodelle. Ein Diagramm wird dann über das Language Server Protocoll (LSP) vom Server zum Client geschickt, welcher dieses anzeigen kann.

KlighD ist ein sehr praktisches und flexibles Werkzeug, welches auch für andere Synthesen in anderen Programmiersprachen eingesetzt werden soll. Tatsächlich verwendet die Königliche Technische Hochschule Schwedens KlighD-Diagramme, um ihre Programmiersprache ForSyDe interaktiv zu visualisieren¹. KLighD wird für verschiedene Diagramme außerhalb von *SCCharts* verwendet, allerdings durch die bisherige Framework Implementierung nur in Java. In vorherigen Implementierungen wurden Teile der API bereits von der Java Abhängigkeit abstrahiert, sodass nun nur noch wenige Teile fehlen, um einen Diagrammservice auch außerhalb von Java mittels der LSP-basierten Kommunikation anzubieten – das automatische Layout. Dementsprechend ist die Idee des Projektes *Diagrams as a Service* wichtige Layouting-Schritte, die KlighD übernimmt auf dem Client zu implementieren. So könnte eine Synthese für eine andere Programmiersprache ein viel leichter LSP-Server sein, der nicht in die komplexe Materie von KlighD eindringt.

In meiner Arbeit am Forschungsprojekt befasste ich mich konkret mit dem *Micro-Layout*-Schritt, welcher auf dem Client implementiert werden muss. Das Micro-Layout ist verantwortlich für die genaue Positionierung und das Aussehen einzelner Graphenelemente. Der Micro-Layout-Schritt besteht aus zwei Teilen, der *Schätzung* und der *Berechnung*, zwischen denen noch das *Macro-Layout* liegt. Diese Schritte bestimmen essentiell die Größen und Abstände zwischen Containern und anderen Elementen im Graphen.

¹<https://github.com/sthaeron/forsyde-devtools?tab=readme-ov-file#visualiser-vscode-extension-installation>

1.1 KlighD Micro-Layout

Micro-Layout Schätzung Die minimalen Größen von den einzelnen Knoten im Graphen werden geschätzt, wobei Unterknoten auch berücksichtigt werden.

Macro-Layout Die Abstände zwischen Knoten und Elementen innerhalb Knoten werden berechnet. Diesen Teil übernimmt *ELK* – ein Werkzeug das Teil von der Eclipse Foundation ist und weltweit entwickelt wird, insbesondere von der RTSys AG.

Micro-Layout Berechnung Die Größen aller Felder innerhalb von Knoten und anderen Elementen werden erneut bestimmt, basierend auf den Größen vom Makro-Layout.

2 Ziele

Das Hauptziel des Forschungsprojektes war es den Micro-Layout-Schritt auf Client Seite zu implementieren. Darüber hinaus gab es mehrere optionale Ziele, die den Zweck hatten die Programmierung einer Synthese zu vereinfachen. Dafür muss das JSON Schema, welches von einer Synthese verwendet wird, angepasst werden, sodass die vom Client erwarteten Diagram Server Protocol Nachrichten vollständig implementiert sind. Dies würde außerdem erheblich die Entwicklung eines zukünftigen Tools erleichtern, welches als Framework dazu dienen soll, um leicht eigene Synthesen zu erstellen in beliebigen Programmiersprachen.

3 Ergebnisse

Die Micro-Layout Schritte konnten in der Zeit zum Teil implementiert werden. Dabei wurden die wichtigsten Features der ursprünglichen KlighD Implementierung auf dem Client umgesetzt. Beispielsweise werden Flächen- und Punkt-platzierte Komponenten nun korrekt gelayoutet. Außerdem wurden viele wichtige Funktionen geschrieben, die auch für andere Micro-Layout Komponenten relevant sind. Eine detailliertere Zusammenfassung meiner Arbeit ist im Forschungstagebuch zu finden.

Die Implementierung involvierte viel Debugging und Tests, und erforderte Recherche über-, sowie Verständnis für KlighD. Somit konnte das JSON Schema nicht mehr in der Zeit angepasst werden. Nichtsdestotrotz wurden für die zukünftige Arbeit am Thema wichtige Fundamente gelegt.

Die einführende Präsentation sowie die Zwischenpräsentation, welche ich über dieses Thema in der RTSys AG vorgeführt hatte, sind zu finden unter https://docs.google.com/presentation/d/1q_nDqA11FoWcPzSdqEaZ6JHt0WQSJ5WqP9HgSchuBlg/.

Weitere Erläuterungen zu KlighD und dem Projekt *Diagrams as a Service* lassen sich finden im korrespondierenden, noch unveröffentlichten, Paper der RTSys AG, sowie in der Dissertation von Niklas Rentz.